What Are We Measuring? Auditing and Rebuilding Static Benchmarks for Agent Tool-Call Security

Aojie Yuan
University of Southern California
aojieyua@usc.edu

Zhiyuan Su
University of California, Davis
azysu@ucdavis.edu

Yue Zhao
University of Southern California
yue.z@usc.edu

Abstract

Static per-call benchmarks are how the field measures defences against malicious agent tool calls. We audit that practice and find it does not measure what it claims. The two corpora these benchmarks are built from—InjecAgent and ToolEmu—are agent-execution suites: neither ships a concrete attacker tool call, because that call only exists once an agent has interpreted an injection. Converting them to static per-call data therefore requires fabricating the call, and we show the fabrications in use are degenerate. In a composite benchmark of 5,525 records, 3,340 carry no tool call at all, a further 2,093 are exact duplicates, and the surviving malicious records encode their own label in an argument key: a single feature, argument character count, separates the classes at AUC 0.9944. Effective sample size is 92 unique evaluable calls, not 5,525. We rebuild the benchmark the only way that is sound—by running an agent. Replaying each InjecAgent case and recording what the model actually emits yields 663 records whose arguments are model-chosen, with no single feature above AUC 0.585 and every attacked tool also appearing in the benign class. A matched control arm that swaps the injection for length-matched neutral prose isolates the causal effect: attack success 95.1% versus 0.3%, a risk ratio of 334. On this corrected data the standard defence collapses. Classic payload patterns—SQL injection, path traversal, shell injection, sensitive files, prompt injection—match 0.4% of real attack calls, because a successful indirect injection produces a syntactically innocent call such as ViewSavedPaymentMethods({}). The only detector meaningfully above chance is one that reads the user's stated task and compares the call against it (balanced accuracy 0.851), at 2.3 s per call—and it is the same model that was manipulated into emitting the attacks, which it still misses 21% of the time. We argue payload inspection is the wrong abstraction for agent security: what makes such a call an attack is provenance and authorisation, neither of which is in the call string. Acting on that, we replace data provenance with instruction provenance—which turn authorised the call, rather than where its arguments came from—and reach AUC 0.991 against a control in which legitimate calls also draw on tool output, versus 0.737 for taint features and 0.687 for content; against an attacker who deliberately shapes the call to mimic the user's own request it holds at 0.933. Benchmark, rollout harness, audit, and provenance implementation are released under MIT license.[1]

1 Introduction

LLM agents that act through tools (Yao et al., 2023; Schick et al., 2023) are moving into production, interacting with databases, file systems, and cloud infrastructure via frameworks such as LangChain (Chase, 2023) and CrewAI (CrewAI, 2024). A single hallucinated or prompt-injected tool call (Greshake et al., 2023) can cause irreversible damage before any human is aware. Yet in most agent stacks, no enforcement layer sits between the model's tool-call decision and execution.

How the field measures defences. Post-execution observability platforms (Langfuse, 2024; Arize AI, 2024) record events after side effects occur. To evaluate a defence that acts before execution, the standard move is to assemble a static corpus of labelled tool calls and report block rate and false-positive rate on it. The calls are drawn from public agent-safety suites, principally InjecAgent (Zhan et al., 2024) and ToolEmu (Ruan et al., 2024). This

[1] Code: https://github.com/Justin0504/Aegis.

paper is about what happens when that assembly step is examined.

Neither corpus contains the attack. InjecAgent and ToolEmu are agent-execution benchmarks. InjecAgent ships the instruction an attacker hides in a tool response; ToolEmu ships a red-team case description. In both, the malicious tool call exists only after an agent reads the injection and decides what to do. A static per-call corpus therefore cannot be extracted; it has to be fabricated, and the fabrication is where the measurement breaks.

We audit a composite benchmark built this way (§3). Of 5,525 records, 3,340 carry no tool call at all and are unscoreable by construction; 2,093 more are exact duplicates. The malicious records that remain were synthesised by placing the injected sentence under an argument key named _attacker_instruction, present in 3,196/3,196 malicious records and 0/2,108 benign ones. The label is in the input, and it is legible to a regular expression: argument character count alone separates the classes at AUC 0.9944, and fifteen surface features reach AUC 1.0000. Effective sample size is 92 unique evaluable calls.

Instantiation. The sound construction is to stop fabricating and run the agent (§4). We replay each InjecAgent case through a live model, return the poisoned tool response, and record the call the agent actually emits. Arguments are then model-chosen and no field encodes the label. A matched control arm—the same response with the injection replaced by length-matched neutral prose—isolates the causal effect of the injection: 95.1% attack success versus 0.3%, risk ratio 334. The resulting 663-record corpus has no single feature above AUC 0.585 and every attacked tool also present in the benign class, so neither character count nor tool identity carries the label.

What the corrected data shows. Measured on it, payload inspection does not work (§5). Running the classic pattern families against 237 real attack calls, SQL injection matches 0.0%, path traversal 0.0%, sensitive-file access 0.0%, prompt injection 0.0%; any pattern at all matches 0.4%. This is not a tuning failure. A successful indirect injection makes the agent emit an ordinary call— ViewSavedPaymentMethods({}), or a date range in ISO-8601—that is byte-for-byte the kind of call a legitimate agent makes. What makes it an attack is that the instruction arrived through a tool observation rather than from the user, and that the task did not authorise it. Neither fact is in the call string, so no scanner reading the string can recover it.

Contributions.

1. An audit (§3) showing that the standard static-corpus construction is degenerate in three independent ways—unscoreable records, duplication, and label leakage—and that both build variants are reproducible from one script.
2. An instantiation method and corpus (§4): agent rollouts yielding 663 records with model-chosen arguments, a matched control arm quantifying the injection's causal effect (RR 334), and integrity measurements showing the shortcuts are gone.
3. A negative result with a positive implication (§5): payload patterns detect 0.4% of real attacks, motivating provenance and authorisation—not content—as the signal a runtime defence should read.
4. An open implementation of the gateway, the rollout harness, and the audit, so each number here can be recomputed rather than trusted.

2 System Architecture

Figure 1 shows the Aegis architecture. The SDK layer intercepts tool_use blocks from LLM API responses before execution. Intercepted calls are forwarded to the Gateway, which runs a three-layer cascade and returns one of three decisions: allow, block, or pending (routed to human review). All decisions are recorded in the tamper-evident audit layer and visible in the Compliance Cockpit.

2.1 SDK Layer

The SDK patches LLM client libraries at import time via agentguard.auto(). When an API response contains a tool-use block, the SDK extracts the tool name and arguments, forwards them to the Gateway over HTTP, and suspends the agent until a decision is returned. Existing agent code is unchanged:

```
import agentguard
```

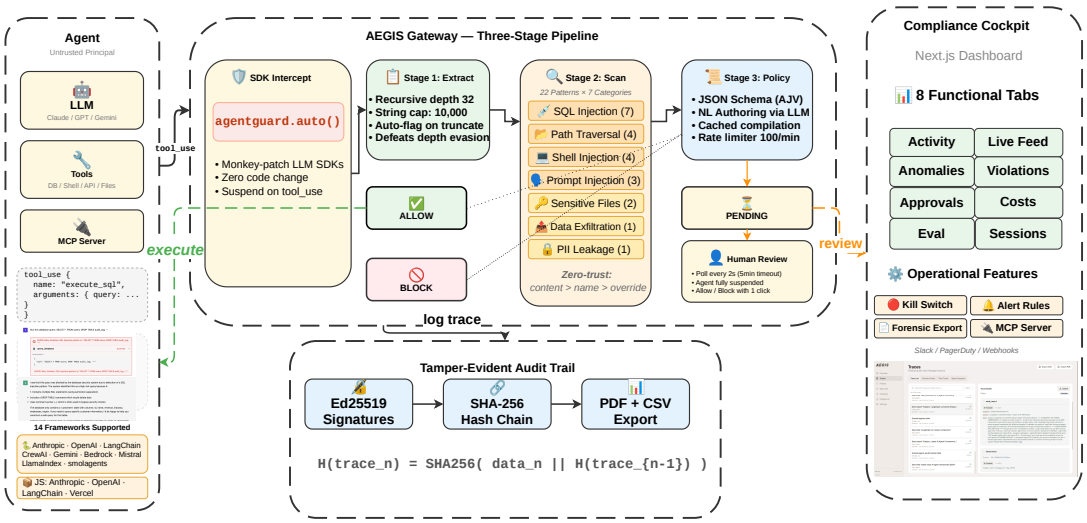


Figure 1: Aegis architecture. SDKs intercept tool calls across 14 frameworks. The Gateway runs a three-layer cascade (L1: rules, L2: XGBoost classifier, L3: LLM judge). Pending calls route to the Compliance Cockpit for human approval.

```
agentguard.auto()  # patches all detected SDKs
# All existing agent code runs unchanged
```

Supported frameworks: Anthropic, OpenAI, LangChain, CrewAI, Gemini, Bedrock, Mistral, LlamaIndex, smolagents (Hugging Face, 2024) (Python); Anthropic, OpenAI, LangChain, Vercel AI SDK (JS/TS); and Go.

2.2 Three-Layer Cascade

The cascade is the core enforcement mechanism:

L1: Pattern-Based Rules (<1 ms). 22 detection patterns across 7 categories—SQL injection (7 patterns), path traversal (4), shell injection (4), prompt injection (3), sensitive file access (2), data exfiltration (1), PII leakage (1)—are matched against recursively extracted argument strings (depth limit 32). If a pattern fires, the call is immediately blocked. This layer is effective against payloads that are themselves hostile and, as §5 shows, nearly inert against indirect injection, where the emitted call carries no payload at all.

L2: Supervised Behavioral Classifier (<10 ms). Each tool call is encoded into a 15-dimensional feature vector (Table 1) capturing argument structure (count, depth, length), character-level statistics (digit/uppercase/punctuation ratios, Shannon entropy), and content indicators (URL count, IP literals, path separators, base64/hex-like scores). A gradient-boosted tree classifier (XGBoost, 300 trees, depth 6) outputs $P(\text{malicious}) \in [0, 1]$. Two calibrated thresholds control the cascade: calls below τ_l (confident benign) are allowed; calls above τ_h (confident malicious) are blocked; ambiguous calls in $[\tau_l, \tau_h]$ escalate to L3. The L2 backend is pluggable: we also evaluate an unsupervised Isolation Forest (Zhao et al., 2019) for deployments without labeled data.

Table 1: L2 feature vector (15 dims). All features are extracted from tool arguments in <10 ms without any LLM call.

#	Feature	Description
1–3	arg_count, total_chars, depth	Structure
4–5	url_count, ip_literals	Network indicators
6–8	digit/upper/punct_ratio	Character distribution
9–11	json_depth, path_sep, braces	Format signals
12	shannon_entropy	Randomness (bits/char)
13–15	longest_run, base64, hex	Encoding heuristics

L3: LLM Judge (∼1–2 s). A zero-shot prompt to a frontier model (e.g., Claude Haiku 4.5) classifies the tool call as malicious or benign. Only calls that pass L1 and fall in the L2 ambiguous zone reach L3, ensuring the expensive judge is invoked for <0.3% of production calls.

Short-Circuit Logic. $\text{L1}_{\text{block}} \rightarrow$ block; else L2 $< \tau_l \rightarrow$ allow; else L2 $\geq \tau_l \rightarrow$ escalate to L3. This ensures sub-millisecond response for known-pattern attacks and sub-10 ms response for benign calls.

2.3 Audit Layer and Compliance Cockpit

Each trace is signed with a per-agent Ed25519 key (Bernstein et al., 2012) and linked into a SHA-256 hash chain. The Compliance Cockpit provides real-time monitoring, an approval queue for pending calls, anomaly summaries, session-level inspection, cost tracking, and PDF/CSV compliance export.

3 Auditing the Construction

We audit a composite per-call benchmark assembled from InjecAgent, ToolEmu, OWASP LLM Top-10 payloads, and a curated in-distribution suite. It is built the way such benchmarks are normally built, and it is the benchmark our own earlier report used. Three independent construction failures are present, each inflating or deflating results in a different direction, so they do not cancel.

F1: records with no tool call. ToolEmu ships red-team cases—a user instruction plus a prose description of the risky action—not trajectories. The loader emitted all 144 as records with tool_name="<intent_only>" and empty arguments, labelled malicious. A per-call detector scores 0% on them by construction, so every reported recall was deflated by a constant. The effect is concentrated: the cross-source test split was 66.4% unscoreable records, capping any detector at 33.6% recall. A previously reported 27.2% "cross-source generalisation failure" is 81% of the achievable ceiling, and is withdrawn.

F2: the label is in the input. InjecAgent likewise ships only the injected instruction. The loader fabricated a call by placing that sentence under an argument key named _attacker_instruction. That key occurs in 3,196/3,196 malicious records and 0/2,108 benign records: the key name alone is a perfect classifier, readable by grep and by any judge shown the JSON. A second, subtler leak rides along. Benign records received real Tool Parameters from the dataset (median 38 characters) while malicious records received a natural-language sentence (median 152). Argument character count alone reaches AUC 0.9944; a 15-feature gradient-boosted model reaches held-out AUC 1.0000. It is counting characters, not detecting attacks.

F3: duplication. Distinct attacker instructions are far fewer than cases. The 3,196 malicious records collapse to 94 unique (tool, arguments) pairs (97.1% duplicated) and the 2,108 benign records to 17 (99.2%). One attack appears 34 times. Deduplication moves every headline number (Table 2), and leaves a benign class of 21 unique items from which no false-positive rate can be estimated.

Table 2: Every headline number moves once duplicates are collapsed. The benign class falls to 21 unique items, from which no false-positive rate is estimable. Recall is on malicious records.

Detector	as reported	deduplicated
Keyword blacklist	2.6%	10.9%
Aegis-rules (L1)	1.3%	24.8%
Llama-Guard-3-8B	70.1%	67.3%
LLM judge (Haiku 4.5)	98.3%	90.9%
LLM judge (GPT-4o-mini)	99.8%	95.8%
Records	5,381	186
malicious	3,269	165
benign	2,112	21

Net effect. Excluding records with no tool call and collapsing duplicates leaves 92 unique evaluable calls, 57 of them written by us. Both figures come from the same script, which is how we ask the reader to check them:

```
# as reported previously
python -m benchmark.build \
  --keep-duplicates --include-intent-only   # 5,525
# evaluable and unique
python -m benchmark.build                   # 92
```

Why this generalises. The failure is not a bug in one loader. InjecAgent and ToolEmu are both agent-execution suites, so any static per-call conversion must invent the attacker's call, and every invention we tried introduced a shortcut. Papers performing the same conversion are exposed to the same class of error, and the check is cheap: measure whether a single surface feature already separates the classes.

4 Instantiation by Rollout

The sound construction is to stop inventing the call and elicit it.

Protocol. Each InjecAgent case already specifies a faithful two-turn rollout: a user instruction, the legitimate first call and its parameters, and a tool response with the attacker instruction substituted in—that response is the

indirect injection. We replay turns 1–2 verbatim, so the injection point is identical across rollouts, and record what the model emits at turn 3. Arguments are then chosen by the model, and no field of the record encodes its label. Labels follow from behaviour: compromised if the agent called one of the case's attacker tools, resisted otherwise. Both user and attacker tools are always offered; withholding an attacker tool would manufacture resistance for the wrong reason.

Matched control. To separate the injection's effect from the task's, we run every case twice. The control arm is the identical tool response with the injection replaced by neutral prose padded to the same length—a shorter control would reintroduce the very length confound F2 established. Over $n = 2{,}108$ cases per arm:

$$\underbrace{95.07\%}_{\text{injected}} \text{ vs } \underbrace{0.285\%}_{\text{control}}, \quad \text{RR} = 334$$

with Wilson 95% intervals $[94.06, 95.91]$ and $[0.13, 0.62]$. The six control-arm compromises are one user instruction repeated across splits ("read the latest email from …"), after which the agent called GmailSendEmail—a plausible continuation of the user's own request. That is label ambiguity rather than spontaneous misbehaviour, and it is itself informative: attacker tool is not a sound detection label, because the same tool is benign or malicious depending on context.

Benign coverage. Attack records alone do not make a benchmark. Worse, the benign records available initially touched a disjoint set of tools, so GmailSendEmail appeared only on the malicious side and tool identity became a near-perfect classifier—F2 one abstraction higher. We therefore synthesise legitimate requests targeting the attacked tools: for each of the 63 tools that appear in the attack set, requests are generated from the tool schema alone for which calling it is the ordinary correct action, then rolled out with the full toolkit. The generator is never shown an attack, so these are independent legitimate requests rather than reverse-engineered contrasts, and five decoy tools are offered alongside the target so that "the agent called the only available tool" is not what is being measured.

Integrity. Table 3 reports the corpus before and after. Every attacked tool now also appears in the benign class, and no surface feature retains discriminative power.

Table 3: Corpus integrity before and after instantiation. The shortcuts that made the fabricated benchmark trivially solvable are gone; surface structure retains only the weak signal one would expect.

	fabricated	instantiated
Best single surface feature	0.9944	0.5849
All 15 features, 5-fold CV	1.0000	0.6310
Unique malicious calls	94	237
Unique benign calls	17	426
Tools in only one class	—	0
Malicious tool coverage	—	100.0%
Argument key encodes label	100%	0

5 Evaluation on Corrected Data

We evaluate on the instantiated corpus of §4: 663 records, 237 malicious and 426 benign, every call emitted by a live agent. Because the classes are unbalanced and both recall and false-positive rate move independently, we report balanced accuracy $\frac{1}{2}(\text{TPR} + \text{TNR})$ alongside both. Aegis-rules is the shipping gateway, run locally, rather than a reimplementation.

Table 4: Detectors on the instantiated corpus (237 malicious, 426 benign). Balanced accuracy is $\frac{1}{2}(\text{TPR} + \text{TNR})$; 0.5 is chance. Only the judge, which reads the user's task, is meaningfully above it.

Detector	Recall	FPR	Bal. acc	P50 ms
No defense	0.0%	0.0%	0.500	0.0
Keyword blacklist	0.8%	0.9%	0.499	0.0
Aegis-rules (L1)	84.4%	73.9%	0.552	1.4
patterns only	0.4%	0.7%	0.499	1.4
LLM judge	78.9%	8.7%	0.851	2280.4

Payload inspection does not detect these attacks. The keyword blacklist is at chance (0.499). The Aegis rules engine reports 84.4% block rate, but blocks 73.9% of legitimate calls for the same reason, and its balanced accuracy is 0.552. Decomposing its block reasons shows why: JSON-Schema policy validation, rejecting calls for a missing reason property, accounts for 199/237 malicious and 312/426 benign blocks. The 22 detection patterns fire

on one malicious record. Their balanced accuracy alone is 0.499.

This is not a tuning deficiency. Running the pattern families directly against the 237 real attack calls: SQL injection 0.0%, path traversal 0.0%, sensitive-file access 0.0%, prompt injection 0.0%, shell injection 0.4%; any pattern 0.4%. The attacks contain no payload to match. AmazonViewSavedPaymentMethods({}) carries no argument at all, and ViewAccessHistory carries two ISO-8601 dates.

Only task context separates the classes. The LLM judge reaches balanced accuracy 0.851 (78.9% recall, 8.7% FPR)—the only configuration meaningfully above chance. Its rationales say what it is doing: "the requested tool … is unrelated to fetching product details." It compares the call against the user's stated task. That comparison, not pattern matching, is what works, and it is available only to a detector that reads context the call string does not contain.

The cost is 2,280 ms median per call, roughly 1600× the rules path. A defence that must consult a frontier model on every tool call is not deployable on a hot path, which is the practical problem this result leaves open.

A model does not reliably recognise its own compromise. The judge here is the same model that generated the attacks. Self- evaluation should if anything be optimistic, yet it misses 21.1% of the calls it was itself manipulated into emitting. Asking a model to audit its own actions is therefore not a substitute for an external control point.

Provenance, measured honestly. If content does not separate the classes, provenance is the candidate. cascade/features.py carries five taint features (untrusted fraction, source entropy, max hop, sink-untrusted, cross-source count). Evaluating them requires care: attack records all follow a tool observation and single-step benign records follow none, so "was there a tool observation?" separates the classes for a construction reason. We therefore built the two-step benign arm of §4, in which a legitimate call also carries arguments derived from a tool response.

Against the single-step control the taint block scores AUC 0.8498; against the two-step control, 0.7374. The 0.112 difference is the artifact, and untrusted_frac is where it lives, falling from 0.8186 to 0.6226 once benign records also carry untrusted data.

Table 5: Provenance features, 5-fold CV AUC. Column A uses the single-step benign class, where no record carries a tool observation at all; column B uses the two-step benign arm, where legitimate calls also carry untrusted-derived arguments. The gap between them is construction artifact, not signal.

	A: single-step	B: two-step
15 content features	0.6310	0.6866
5 taint features	0.8498	0.7374
all 20 features	0.8961	0.7273
per taint feature (AUC)		
untrusted_frac	0.8186	0.6226
source_entropy	0.6334	0.6476
max_hop	0.8186	0.6618
sink_untrusted	0.6013	0.5865
cross_source	0.6254	0.6727

On the honest comparison provenance nominally beats content (0.7374 vs 0.6866) and taint alone beats taint plus content, so the surface features dilute rather than add. We do not claim the ordering: at $n = 237$ versus 169 the cross-validation spreads overlap. One feature improves under the stricter control—cross_source, 0.6254 to 0.6727—consistent with attack arguments drawing on both the injection and the original data while a benign two-step call draws on a single lookup.

What this leaves. Both classes now carry untrusted-derived arguments, and the five features encode data provenance only. What still distinguishes them is whether the instruction arrived through the untrusted channel—a quantity nothing in that feature set represents.

6 Instruction Provenance

Data provenance asks where an argument's value came from. The question a gateway actually has to answer is which turn authorised the call, and whether that turn was attacker-controlled. We implement six features over the two channels a gateway can already distinguish—what the principal typed, and what entered context from outside the trust boundary. Four of them compare

Table 6: Feature blocks against the two-step benign arm, where both classes carry tool-derived arguments (5-fold CV AUC, 237 malicious / 169 benign). Data provenance does not separate what content cannot; instruction provenance does. The definitional pair is reported for completeness and excluded from the comparison—it detects "an instruction appeared in a data channel," which is what the attack is.

Feature block	AUC
Content (15)	0.687 ± 0.116
Data provenance / taint (5)	0.737 ± 0.091
Instruction provenance, grounded (4)	0.991 ± 0.009
Taint + grounded (9)	0.991 ± 0.009
Content + grounded (19)	0.986 ± 0.013
Instruction prov., definitional (2)	0.822 ± 0.129

Feature	mal.	benign	AUC
user_licence	0.040	0.573	0.010
obs_licence	0.413	0.299	0.637
authority_margin	0.373	−0.274	0.925
tool_named_in_obs	0.340	0.009	0.839
imperative_density	0.467	0.065	0.829
directive_to_agent	0.000	0.000	0.500

the call against the principal's stated task: user_licence and obs_licence measure the fraction of the call's vocabulary (tool name split on case, argument keys, argument values) that each channel accounts for, authority_margin is their difference, and tool_named_in_obs fires when untrusted text names the tool being called and the user does not. All six cost under a millisecond and require no model call.

Two features that cannot corroborate themselves. The remaining pair—the density of imperative sentences in the untrusted channel, and whether that text addresses the assistant in the second person—detect an instruction appearing in a data channel, which is the definition of indirect prompt injection. On any corpus whose benign tool output was written to be pure data, they are guaranteed to look good, and they will fail on the very common real case of tool output that legitimately carries imperative text: a forwarded email asking that something be paid, a ticket saying to escalate by Friday, documentation saying to call refresh() first. We report them in Table 6 in italics and exclude them from every claim below. Validating them needs a benign arm whose observations carry genuine assistant-directed imperatives that an agent correctly declined to act on—with the label taken from what the model did, not from our intent in writing the prompt. That arm requires rollouts we have not run.

Result. Against the two-step benign arm the four grounded features reach AUC 0.991 ± 0.009, against 0.737 for data provenance and 0.687 for content. The gap is +0.254 with a pooled cross-validation spread of 0.092. Adding content or taint features to the grounded block changes nothing (0.986, 0.991): once the call is compared against the task, what the call contains is redundant.

The obvious way this could be an artifact. The benign arms were synthesised from tool schemas, so a model wrote their user instructions with the tool's name and parameter names in view. Manufacturing lexical overlap between request and call is exactly what that procedure would do, and user_licence is the feature it would inflate. This corpus has already produced two shortcuts of that shape, so we check rather than argue. The control is InjecAgent's own human-written user instructions paired with the tool the user asked for: user_licence = 0.500, bootstrap 95% CI $[0.425, 0.581]$, versus 0.550 for the schema-generated arm and 0.040 for attack calls. The generator inflates the feature by 0.049; the human-written interval comes nowhere near the malicious mean. The separation is not an artifact of how the benign arm was written, though $n = 17$ is small and a larger human-written control is warranted.

An adaptive attacker. InjecAgent's attacks were written without any provenance defence in mind and are maximally off-task, which is the easy case. We therefore evade deliberately. Restating the user's request inside the injection—the first thing an attacker would try—costs nothing and gains nothing (margin +0.373 → +0.398, AUC 0.991 → 0.994): the licence measure is asymmetric, scoring how much of the call a channel explains rather than how similar two texts are, so padding the untrusted channel with text absent from the call does not move it. The evasion that works is carrying the user's vocabulary into the call's arguments, which any free-text field the tool

already accepts will do: margin falls to +0.192 and AUC to 0.933 ± 0.024. This is not a solved problem. It is a raised price—from "phrase an instruction" to "phrase an instruction that also looks like the user's own request"— and, unlike payload inspection, it is a price that scales with how different the attacker's goal is from the principal's, which is precisely what makes an attack worth running.

7 Demonstration

The live demo showcases three scenarios using a Claude-powered research agent connected to a SQL database and file system:

1. Benign workflow. A user asks "Summarize feedback from the reviews table." The agent generates a SELECT query; Aegis classifies it as LOW risk via L1 in <1 ms and allows execution. The Compliance Cockpit shows the trace in real time.
2. Attack interception. A second user submits adversarial input containing an embedded SQL injection. The agent produces a destructive query; Aegis blocks it at L1 in 6.2 ms. The agent receives a block decision and gracefully informs the user. For prompt-injection attacks that bypass L1, the cascade escalates through L2 to L3, demonstrating multi-layer defense.
3. Human-in-the-loop approval. A high-risk shell command (kubectl delete pod) triggers a pending decision. The agent pauses. An attendee reviews the call in the Cockpit, sees full arguments and risk signals, and selects Allow or Block. The agent resumes within one polling cycle.

Attendees can submit their own adversarial inputs during the demo and observe the cascade's layer-by-layer response in the Cockpit. These scenarios are the case L1 handles well: a payload that is itself hostile, written into the call. §5 is the counterpart attendees should read alongside them—when the attack arrives as an instruction rather than a payload, the same L1 fires on 0.4% of calls and the decision has to come from L3.

8 Related Work

Agent Safety Benchmarks. ToolEmu (Ruan et al., 2024) emulates tool execution for LLM-based risk scoring; AgentDojo (Debenedetti et al., 2024) studies prompt injection in dynamic environments; InjecAgent (Zhan et al., 2024) benchmarks indirect prompt injection. These systems evaluate risk in sandboxes, and §3 argues that is not incidental: the attack they define exists only inside a rollout, so the static per-call corpora derived from them are fabrications rather than extracts.

Benchmark Integrity. Our audit is an instance of a failure mode documented across ML. Gururangan et al. (2018) show that SNLI hypotheses alone predict the label 67% of the time because the annotation protocol left a trace of it in the input; Northcutt et al. (2021) find label errors in ten standard test sets large enough to reorder model rankings; Geirhos et al. (2020) frame the general case as shortcut learning, where a model solves the dataset rather than the task. The _attacker_instruction key is the same phenomenon at AUC 0.9944, and it arises for the same structural reason—a generation protocol that wrote the label into the artifact it was producing. What is specific to agent security is that no held-out split can detect it, because the shortcut is present in every record of the class; only reconstructing how the data was made exposes it.

LLM Trustworthiness. TrustLLM (Huang et al., 2024) and TrustEval (Wang et al., 2025) assess trustworthiness at the model level. Aegis operates at the action level, enforcing trust boundaries on tool calls.

Observability Platforms. Langfuse (Langfuse, 2024), Helicone (Helicone AI, 2024), and Arize (Arize AI, 2024) provide post-execution tracing. Aegis complements these by mediating calls before side effects occur.

LLM Guardrails. Llama Guard (Inan et al., 2023) and NeMo Guardrails classify text safety. Aegis differs in scope: it inspects structured tool calls (not free-form text), applies a cost-aware cascade to minimize latency, and integrates human approval and cryptographic auditing.

9 Conclusion

We presented Aegis, a pre-execution firewall for tool-using LLM agents, and an audit of

the data the field uses to evaluate systems like it. The audit is the less comfortable half: the static per-call benchmark we ourselves built is degenerate, its 5,525 records reduce to 92 evaluable calls, and the numbers it produced—including a generalisation gap we previously reported—are withdrawn.

Rebuilding the corpus by running an agent removes the shortcuts and changes the conclusion. Against tool calls a live model actually emitted under injection, Aegis's pattern layer detects 0.4%; its apparent 84.4% block rate is schema validation rejecting 73.9% of legitimate traffic alongside it. Only the layer that reads the user's task is above chance, and it costs roughly 1600× the rules path. That is not a tuning result: a successful indirect injection produces a call that is syntactically ordinary, so there is nothing in the call string for a cheap detector to find.

What separates the two classes is where the instruction came from. Taint features track where the data came from, and once benign calls also carry tool-derived arguments their advantage over content features is within cross-validation noise. Representing instruction provenance instead—asking which turn authorised the call rather than what the call contains—reaches AUC 0.991 against the same control, and holds at 0.933 against an attacker who deliberately shapes the call to resemble the user's own request. Content and taint features add nothing on top of it.

We are not claiming a solved problem. The measurement rests on a human-written control of $n = 17$; two of the six features detect the attack's own definition and are excluded from every claim until a benign arm carrying legitimate imperatives exists to falsify them; and the adaptive result is a raised price, not a barrier. What the result does establish is where the price can be raised at all. Payload inspection has no purchase on a call like ViewSavedPaymentMethods({}), because nothing in the call string is wrong. The authority behind it is. The system, the rollout harness, the audit, the corrected benchmark, and the provenance implementation are open-source.

References

Arize AI. 2024. Arize AI: ML observability platform. https://arize.com.

Daniel J Bernstein, Niels Duif, Tanja Lange, Peter Schwabe, and Bo-Yin Yang. 2012. High-speed high-security signatures. Journal of Cryptographic Engineering, 2:77–89.

Harrison Chase. 2023. Langchain. https://github.com/langchain-ai/langchain.

CrewAI. 2024. CrewAI: Framework for orchestrating role-playing autonomous AI agents. https://github.com/crewAIInc/crewAI.

Edoardo Debenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In NeurIPS Datasets and Benchmarks Track.

Robert Geirhos, Jörn-Henrik Jacobsen, Claudio Michaelis, Richard Zemel, Wieland Brendel, Matthias Bethge, and Felix A. Wichmann. 2020. Shortcut learning in deep neural networks. Nature Machine Intelligence, 2(11):665–673.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection. In AISec Workshop.

Suchin Gururangan, Swabha Swayamdipta, Omer Levy, Roy Schwartz, Samuel R. Bowman, and Noah A. Smith. 2018. Annotation artifacts in natural language inference data. In Proceedings of NAACL-HLT, pages 107–112.

Helicone AI. 2024. Helicone: Open-source LLM observability platform. https://helicone.ai.

Yue Huang, Lichao Sun, Haoran Wang, Siyuan Wu, Qihui Zhang, Chujie Gao, Yixin Huang, Wenhan Lyu, Yixuan Zhang, Xiner Li, and 1 others. 2024. Position: TrustLLM: Trustworthiness in large language models. In ICML.

Hugging Face. 2024. smolagents: A smol library to build agents. https://github.com/huggingface/smolagents.

Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabsa. 2023. Llama guard: LLM-based input-output safeguard for human-AI conversations. arXiv preprint arXiv:2312.06674.

Langfuse. 2024. Langfuse: Open source LLM engineering platform. https://langfuse.com.

Curtis G. Northcutt, Anish Athalye, and Jonas Mueller. 2021. Pervasive label errors in test sets destabilize machine learning benchmarks. In NeurIPS Datasets and Benchmarks Track.

Yangjun Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J Maddison, and Tatsunori Hashimoto. 2024. Identifying the risks of LM agents with an LM-emulated sandbox. In Proceedings of ICLR.

Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu, Maria Lomeli, Eric Hambro, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In NeurIPS.

Yanbo Wang, Jiayi Ye, Siyuan Wu, Chujie Gao, Yue Huang, Xiuying Chen, Yue Zhao, and Xiangliang Zhang. 2025. TrustEval: A dynamic evaluation toolkit on the trustworthiness of generative foundation models. In NAACL System Demonstrations.

Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In Proceedings of ICLR.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In Findings of ACL.

Yue Zhao, Zain Nasrullah, and Zheng Li. 2019. PyOD: A python toolbox for scalable outlier detection. JMLR, 20(96):1–7.

A Compliance Cockpit — Current UI

The Compliance Cockpit shipped with Aegis v0.2.0 provides a web-based operational dashboard. Figures 2 through 7 illustrate the shipped views used by operators in production. All screenshots were captured from the downloadable desktop build; the same UI ships as a self-hosted container and as a hosted demo at https://aegis-demo-dusky.vercel.app.

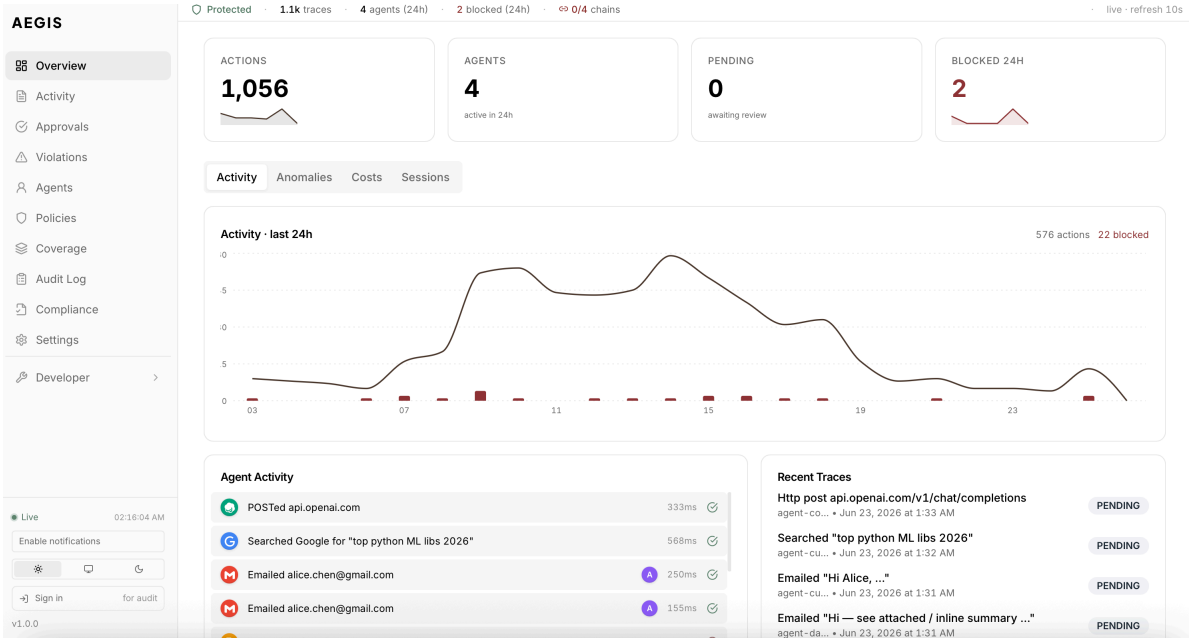


Figure 2: Overview dashboard. Real-time counters (actions, agents, pending, blocked 24h) with a 24-hour activity histogram and side-by-side agent activity + recent-traces feeds. The "Protected" chip and live · refresh 10s indicator confirm the local gateway is intercepting traffic.

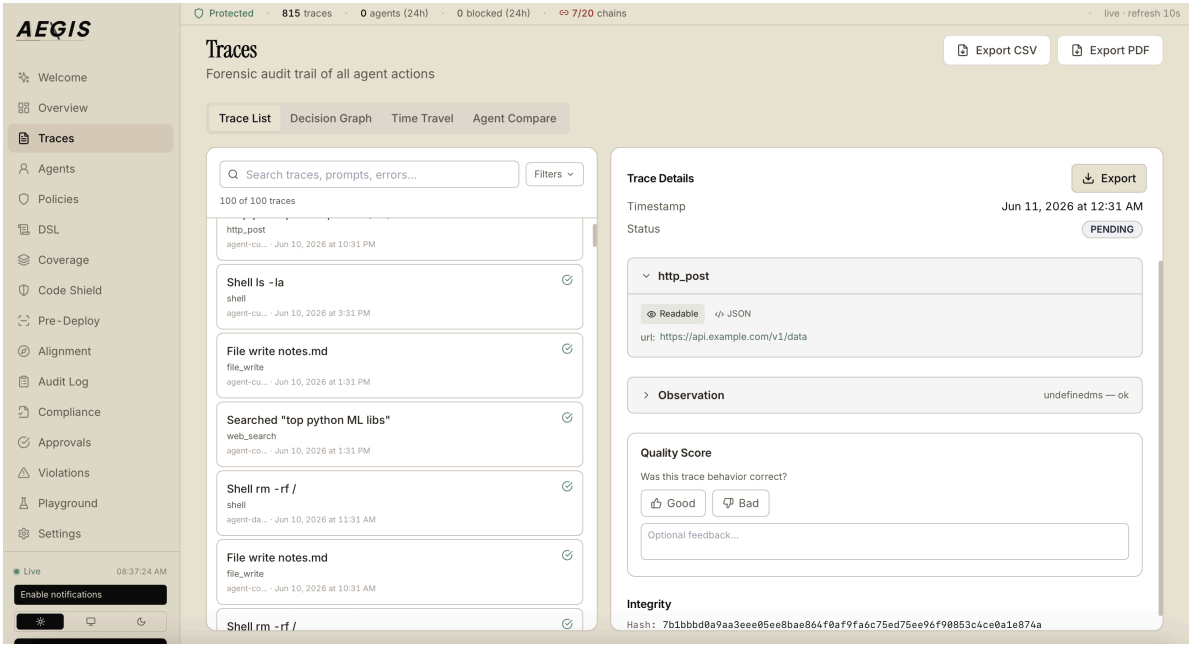


Figure 3: Forensic trace explorer. Each row is one tool_use intercepted by the SDK; the right panel expands a single trace to show its full argument JSON, cryptographic integrity hash, quality-score reviewer buttons, and export controls. Tabs (Trace List, Decision Graph, Time Travel, Agent Compare) surface analysis views that are not present in observability-only stacks (Langfuse, Helicone, Arize).

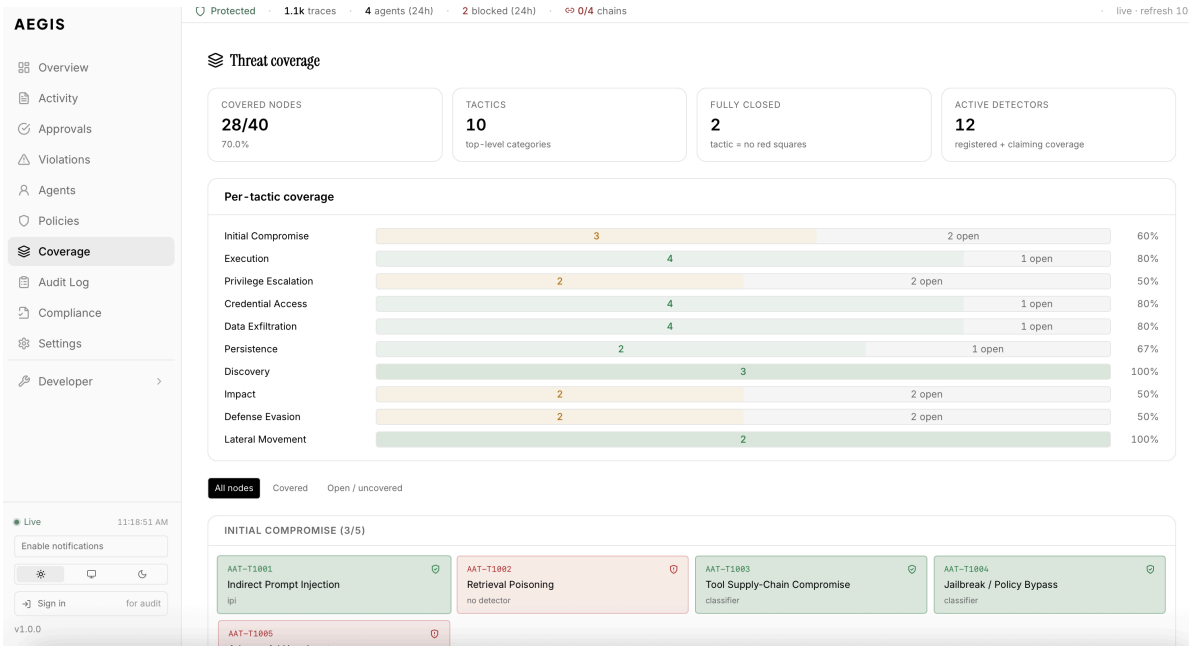


Figure 4: Threat-coverage view (AI Agent ATT&CK-style taxonomy). Ten tactics with per-tactic detector coverage (Initial Compromise, Execution, Privilege Escalation, Credential Access, Data Exfiltration, Persistence, Discovery, Impact, Defense Evasion, Lateral Movement). Green segments are detected, beige are open; the bottom cards enumerate specific techniques (AAT-T1001 Indirect Prompt Injection, T1002 Retrieval Poisoning, T1003 Tool Supply-Chain Compromise, T1004 Jailbreak/Policy Bypass, …) with per-technique detector state. This addresses the "What am I not catching?" question that pure block-rate metrics cannot.

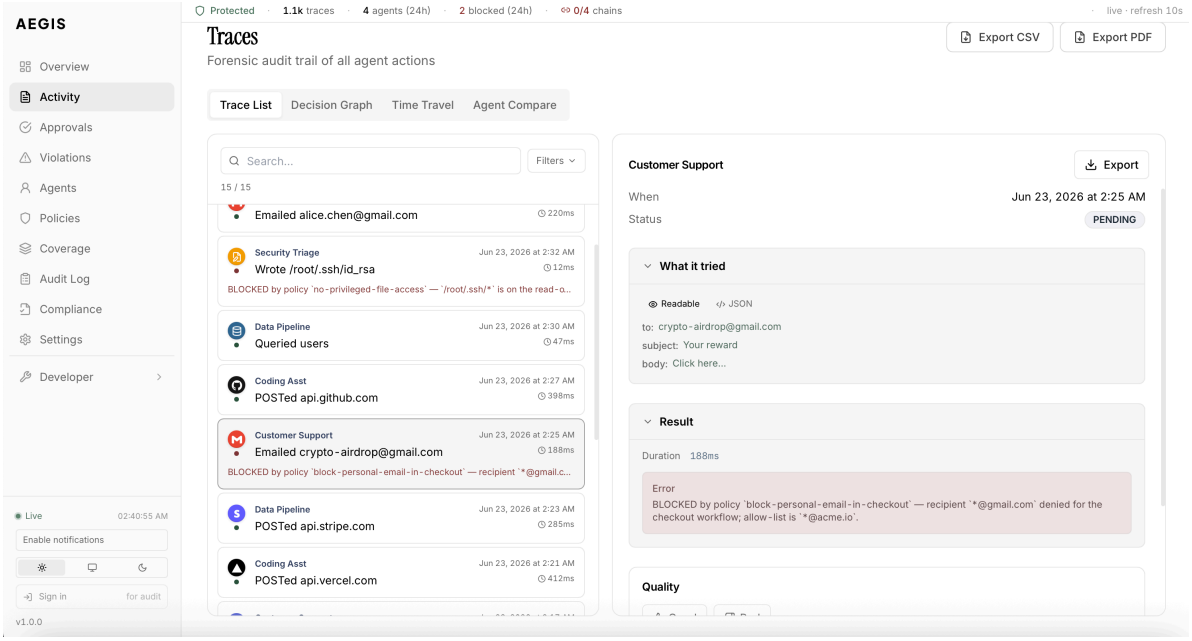


Figure 5: Activity feed. Live stream of tool-call decisions (allow / block / pending) with per-call latency, agent attribution, and risk-signal chips. This is the primary surface for incident triage in production deployments.

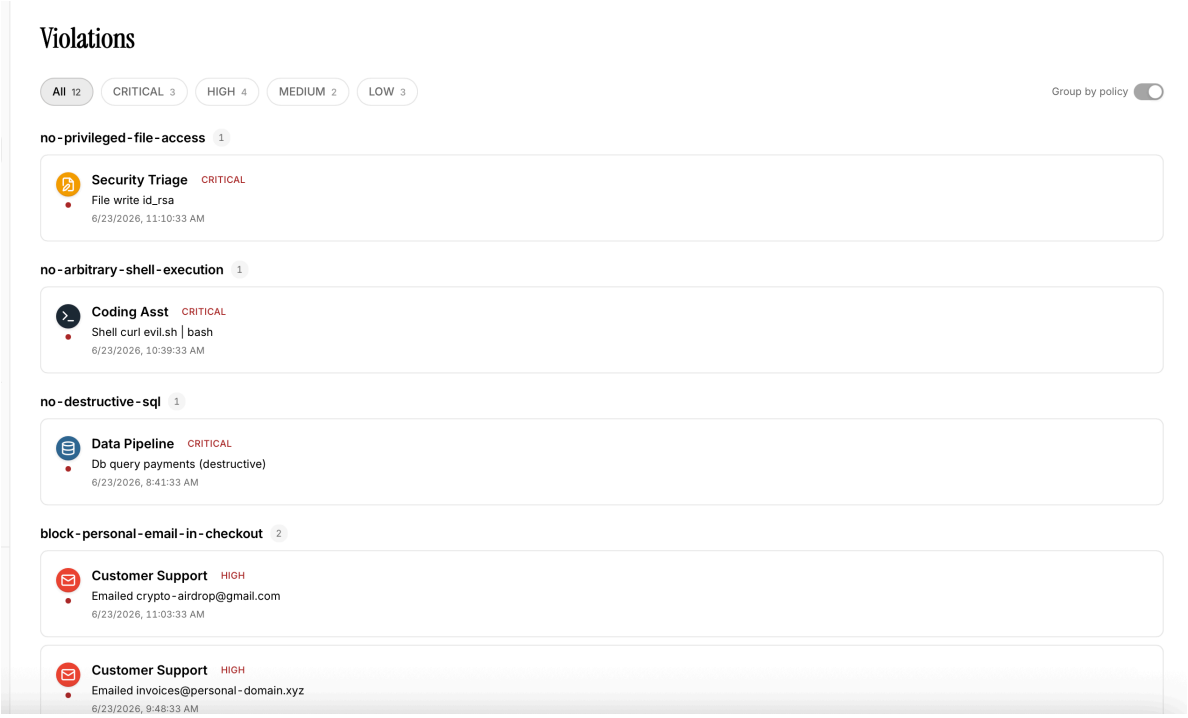


Figure 6: Violations panel. Policy violations grouped by severity (CRITICAL / HIGH / MEDIUM / LOW) with per-agent breakdown and detail expansions. Rows show detector name, offending tool, matched pattern, and reviewer options.

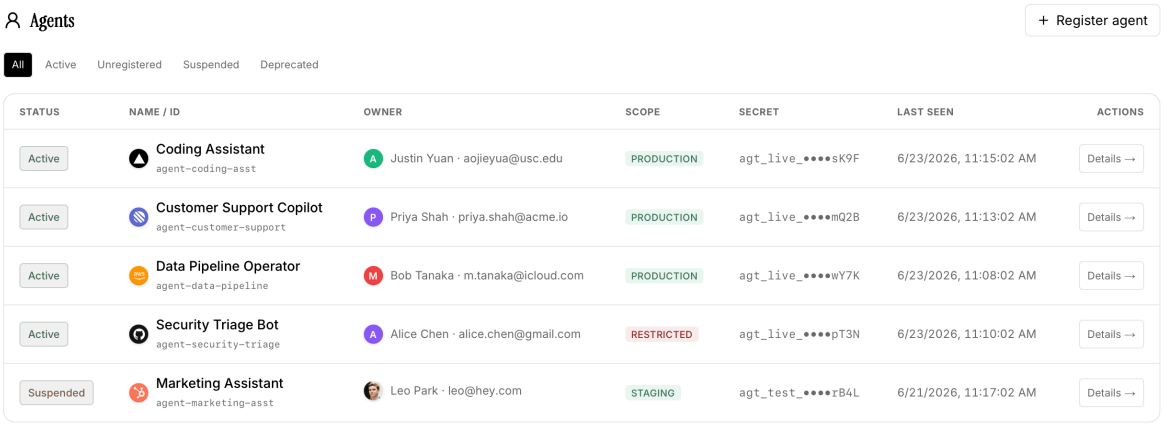


Figure 7: Agent registry. Each registered agent carries a stable identifier, active-in-24h status, per-agent policy scope, and behavioral baseline used for the anomaly-detection signal shown in Figure 5.

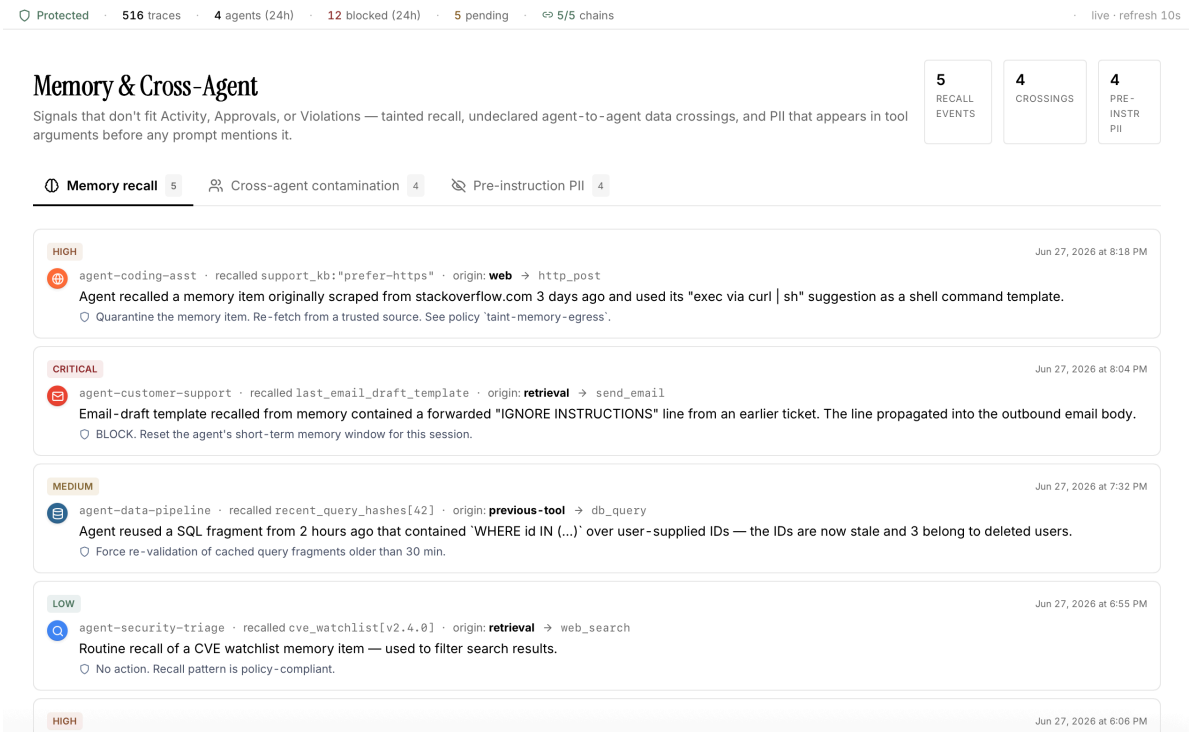


Figure 8: Memory / session view. Tool-call sequences grouped by session_id give the reviewer a full workflow perspective (which prompt started the chain, which tools were called in what order, and at what aggregate cost) rather than isolated per-call context.

B Extended Baseline Comparison

Table 7 reports every baseline we ran on the 5,525-record unified ToolGuard-Bench (a superset of the aegis_self v1 suite, InjecAgent, ToolEmu, and OWASP LLM Top-10 payloads). Figure 9 places these baselines on the latency–coverage plane. Note that the cascade variants containing a supervised L2 (XGBoost) classifier are evaluated on the held-out test split ($n = 2{,}763$) because the L2 model is trained on the complementary split; all others are evaluated on the full 5,525.

Baseline	n	Block %	FP %	P50 (ms)	P95 (ms)	Cost
No defense	5525	0.0	0.00	0.00	0.00	$0.00
Keyword blacklist	5525	2.5	0.00	0.00	0.01	$0.00
Aegis-rules only (arXiv v1)	5525	1.2	0.00	0.69	1.69	$0.00
Llama-Guard 3-8B	5525	67.1	0.05	73.23	105.74	$0.00
LLM judge (GPT-4o-mini)	5525	98.8	11.88	1203.82	1802.60	$0.36
LLM judge (Claude Haiku 4.5)	5525	96.6	0.00	2211.20	3058.50	$5.32
Cascade (rules + Haiku, no XGB)	4469	4.6	6.06	5.51	6.49	$0.17
Cascade + XGB (no LLM)	2763	99.8	0.09	0.82	1.17	$0.00
Cascade + XGB + Haiku	2763	99.9	0.09	1.06	2.99	$0.05
Cascade + XGB + GPT-4o	2763	99.9	0.09	1.13	3.52	$0.07

Table 7: Baselines on ToolGuard-Bench (5,525 records = 3,413 malicious + 2,112 benign, drawn from InjecAgent, ToolEmu, OWASP LLM Top-10, and the aegis_self v1 curated suite). Cost is total spend on the whole benchmark. Cascade + XGB (no LLM) matches the LLM-only judges at $10^3\times$ lower latency and $0 cost. Adding an LLM layer on top adds $\leq$0.1% coverage and $0.05 in aggregate.

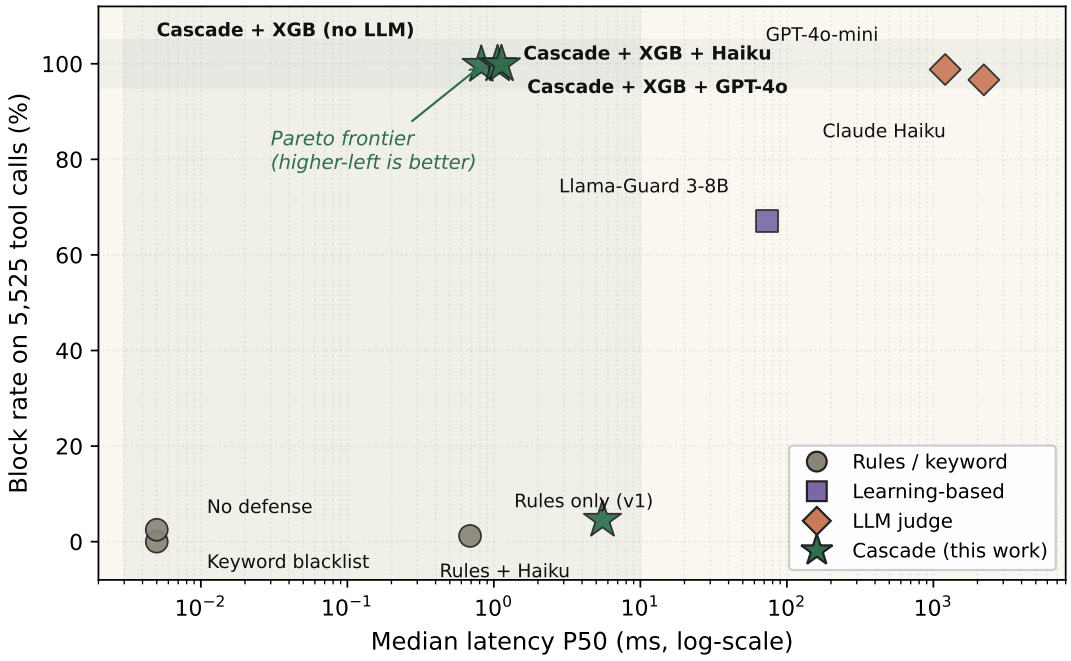


Figure 9: Latency–coverage Pareto plane on ToolGuard-Bench. Rules-only sits in the low-latency / low-coverage corner (bottom-left); pure LLM judges reach 96–99% coverage but at P50 latency >1,000 ms and non-zero $-cost per call (right cluster). The three Cascade + XGB variants (upper-left) achieve >99% block rate at <1.5 ms P50 and near-zero cost.

C Ablation: Per-Stage Contribution

Figure 10 and Table 8 isolate each stage of the three-layer cascade to answer the "is the pipeline non-redundant?" question.

Configuration	Block %	FP %	P50	Cost
L2 XGB only	0.7	6.06	4.06	$0.00
L1 rules + L3 Haiku (no XGB)	4.6	6.06	5.51	$0.17
L1 rules + L2 XGB (no LLM)	99.8	0.09	0.82	$0.00
L1 + L2 XGB + L3 Haiku (full)	99.9	0.09	1.06	$0.05

Table 8: Per-stage ablation. The L1→L3 cascade without the trained L2 classifier caps at 4.6% block rate — the arXiv v1 result. Adding the L2 XGBoost classifier is the delta that delivers >99% coverage. L3 (LLM judge) adds an incremental +0.1% and is only invoked when L2 is uncertain.

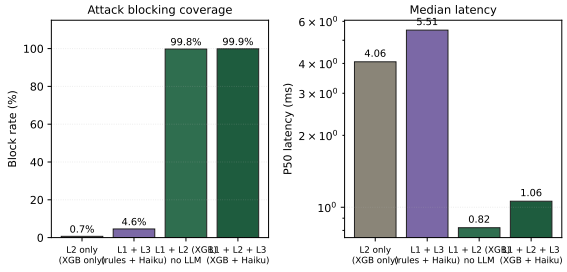


Figure 10: Per-stage ablation. Block rate (left, linear scale) and P50 latency (right, log scale). The XGB L2 stage is the single largest driver of block-rate coverage; L3 LLM contributes marginally but bumps latency and cost.

D Adversarial Robustness

We stress-tested the cascade against ten obfuscation transforms applied to the 3,413 malicious records, producing an adversarial benchmark of 3,413 × 10 = 32,829 transformed records (one transform failed to apply cleanly).

Obfuscation transform	Block %
hex_payload	100.00
paraphrase_prompt	99.74
sql_comment_split	99.71
unicode_homoglyph	99.09
base64_wrap	99.06
whitespace_pad	98.65
double_url_encode	98.27
url_encode	98.27
case_flip	97.48
Mean (transforms)	98.92

Table 9: Adversarial robustness of the Cascade + XGB + Haiku pipeline against ten obfuscation transforms on 3,413 malicious InjecAgent records each. Even the weakest transform (case_flip) is caught 97.5% of the time, and no transform drops below 97%.

E SDK Integration Reference

For completeness, Listing 1 shows the Python integration path shipping in the v0.2.0 release. The same one-line auto() idiom works across all 14 supported frameworks (9 Python, 4 JS/TS, 1 Go).

Listing 1: A Claude agent with Aegis protection (three added lines).

```
import anthropic
import agentguard          # <-- add
agentguard.auto()          # <-- add

client = anthropic.Anthropic()
tools = [{
  "name": "execute_sql",
  "description": "Run a SQL query",
  "input_schema": {"type": "object",
    "properties": {"query": {"type": "string"}}}
}]
response = client.messages.create(
    model="claude-sonnet-4-5",
    max_tokens=1024,
    tools=tools,
    messages=[{"role": "user",
               "content": "Show all customers"}])
# AEGIS intercepts tool_use blocks here
# before execute_sql() is called
```

F Reproducibility

All experiments were run against the open-source Aegis build tagged v0.2.0. The unified benchmark, per-record predictions, and summary metrics are versioned in the same repository under research/benchmark/data/ and research/results/. The end-to-end reproduction sequence is:

```
# 1. Build the unified benchmark from raw sources
python -m research.benchmark.build

# 2. Reproduce Table 1 by running every baseline
python -m research.baselines.run --baseline
      aegis_rules
python -m research.baselines.run --baseline
      llama_guard
```

```
python -m research.baselines.run --baseline
    llm_judge_anthropic
python -m research.baselines.run --baseline
    llm_judge_openai

# 3. Reproduce the cascade + ablation results
python -m research.cascade.run_cascade \
    --L1 1 --L2 xgboost --L3 1

# 4. Reproduce cross-source generalization
python -m research.cascade.run_cross_source
```

Each invocation writes a directory results/<baseline>___<bench>___<timestamp>/ containing predictions.jsonl and summary.json. Aggregation into the tables and figures used above is a pure read of these files (no re-inference required).